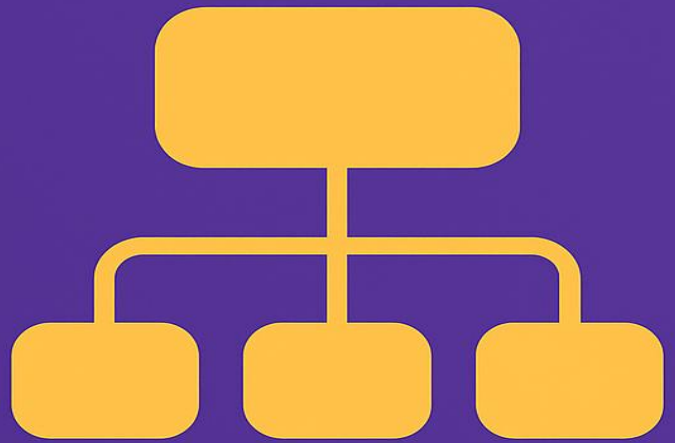
TEMPLATE METHOD



DESIGN PATTERN

Creational (5)

-  Factory
- Abstract Factory
-  Singleton
- Prototype
- Builder

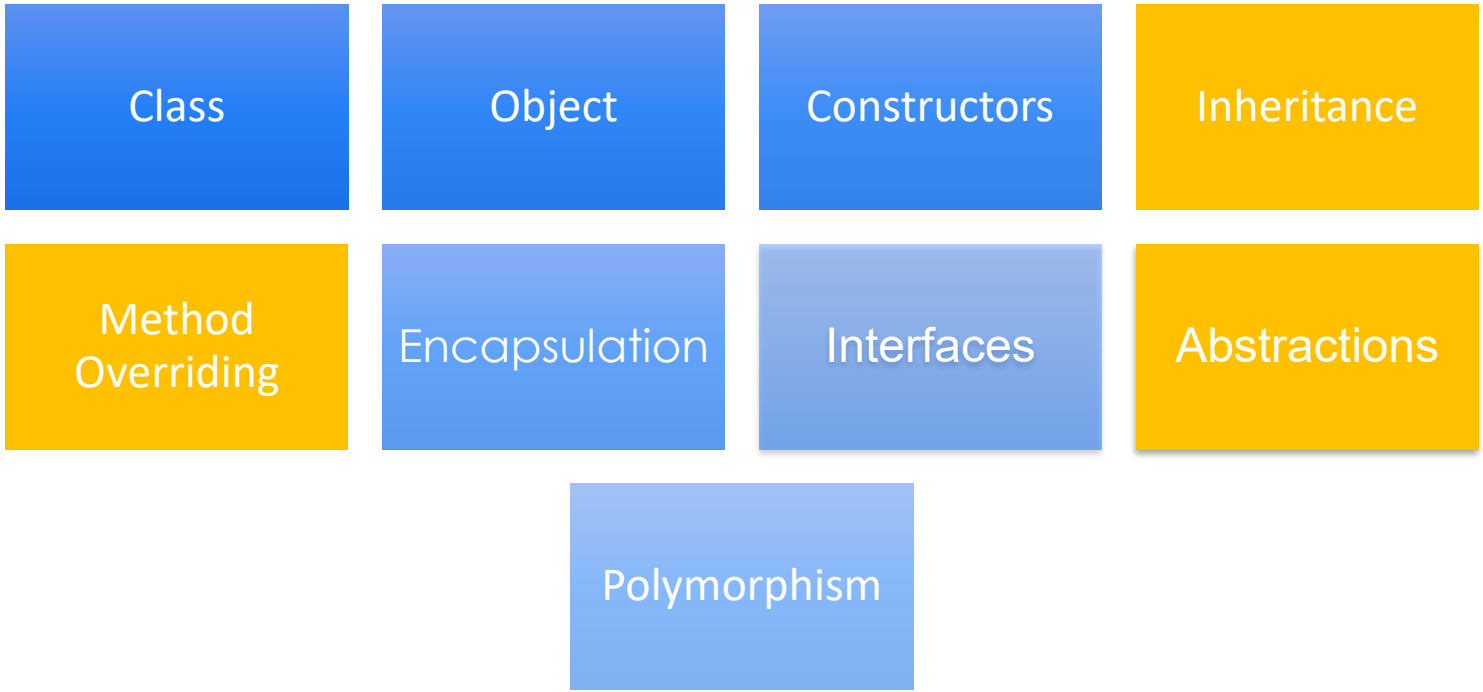
Structural (7)

-  Adapter
- Bridge
- Composite
-  Decorator
-  Facade
- Flyweight
-  Proxy

Behavioral (11)

- Chain Of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
-  Template
- Visitor

OOP Principals



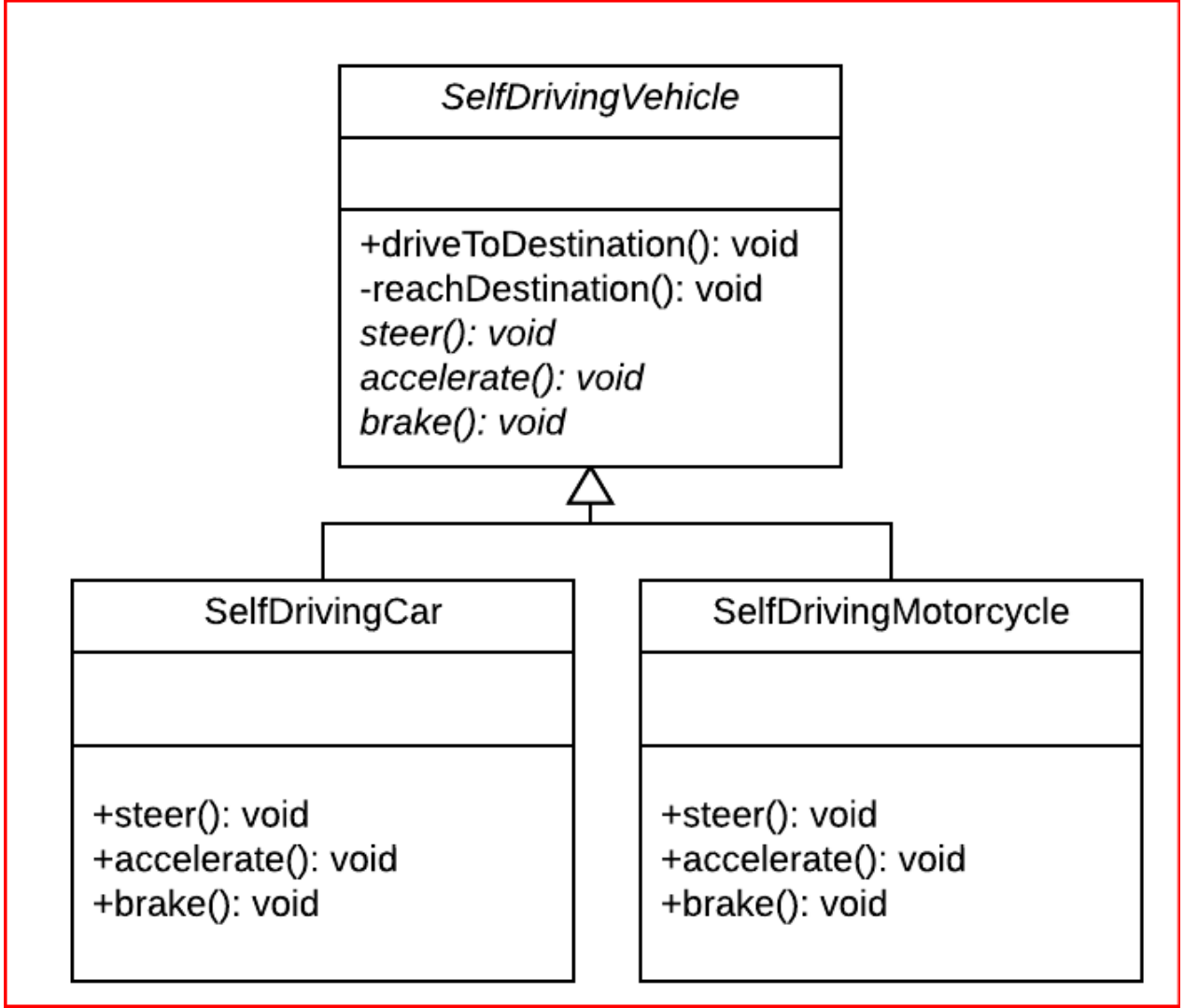
Template Method Design Pattern

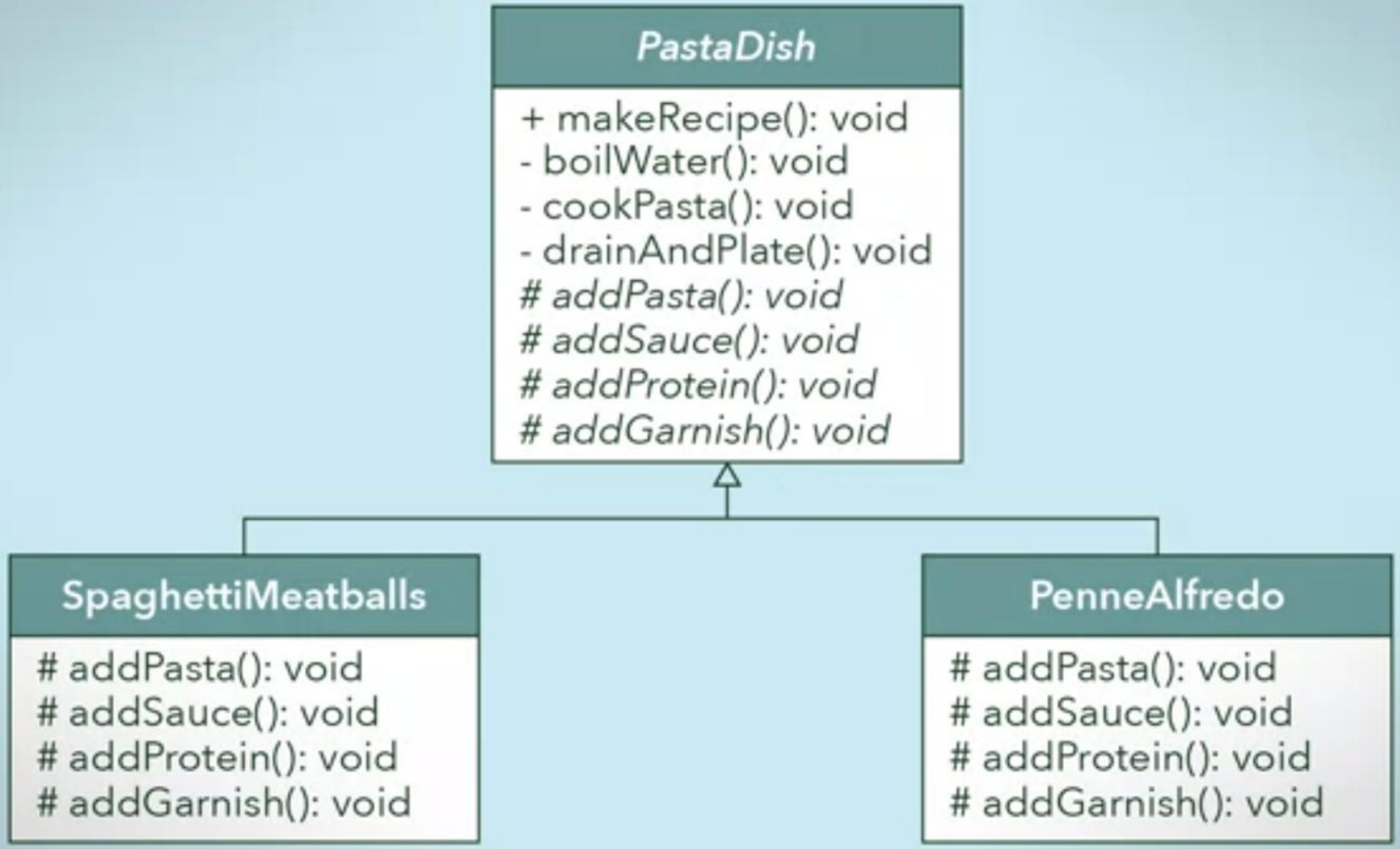
Behavioral patterns also focus on how independent objects work towards a common goal.

The template method defines an algorithm's steps deferring the implementation of some steps to subclasses.

It is a behavioral design pattern and is concerned with the assignment of responsibilities.

Template Method Pattern defines the skeleton of an algorithm in the superclass but allows subclasses to override specific steps of the algorithm without changing the structure.


SelfDrivingVehicle
+driveToDestination(): void
-reachDestination(): void
steer(): void
accelerate(): void
brake(): void
SelfDrivingCar
+steer(): void
+accelerate(): void
+brake(): void
SelfDrivingMotorcycle
+steer(): void
+accelerate(): void
+brake(): void


PastaDish
+ makeRecipe(): void
- boilWater(): void
- cookPasta(): void
- drainAndPlate(): void
addPasta(): void
addSauce(): void
addProtein(): void
addGarnish(): void
SpaghettiMeatballs
addPasta(): void
addSauce(): void
addProtein(): void
addGarnish(): void
PenneAlfredo
addPasta(): void
addSauce(): void
addProtein(): void
addGarnish(): void

```
public abstract class PastaDish {

    public final void makeRecipe() {
        boilWater();
        addPasta();
        cookPasta();
        drainAndPlate();
        addSauce();
        addProtein();
        addGarnish();
    }

    protected abstract void addPasta();
    protected abstract void addSauce();
    protected abstract void addProtein();
    protected abstract void addGarnish();

    private void boilWater() {
        System.out.println("Boiling water");
    }
    …
}
```

Recap: Interface vs Abstract

Aspect	Interface	Abstract Class
Purpose	Defines a contract (what a class must do).	Provides a base class with common behavior (what a class is).
Methods	Only method signatures (no implementation, except default/static methods in some languages).	Can have both abstract methods (no body) and concrete methods (with body).
Fields / Variables	Cannot have instance variables (only constants).	Can have instance variables and state.
Inheritance	A class can implement multiple interfaces.	A class can inherit only one abstract class.
Use Case	When different classes must follow the same rules but are otherwise unrelated.	When classes share a common base and behavior, but also need to enforce some methods.



Final – Cannot be overwritten at the sub classes.

```
public class SpaghettiMeatballs
extends PastaDish {
      protected void addPasta() {
            System.out.println("Add
            spaghetti");
      }
      protected void addProtein() {
            System.out.println("Add
            meatballs");
      }
      protected void addSauce() {
            System.out.println("Add
            tomato sauce");
      }
      protected void addGarnish() {
            System.out.println("Add
            Parmesan cheese");
      }
}
```

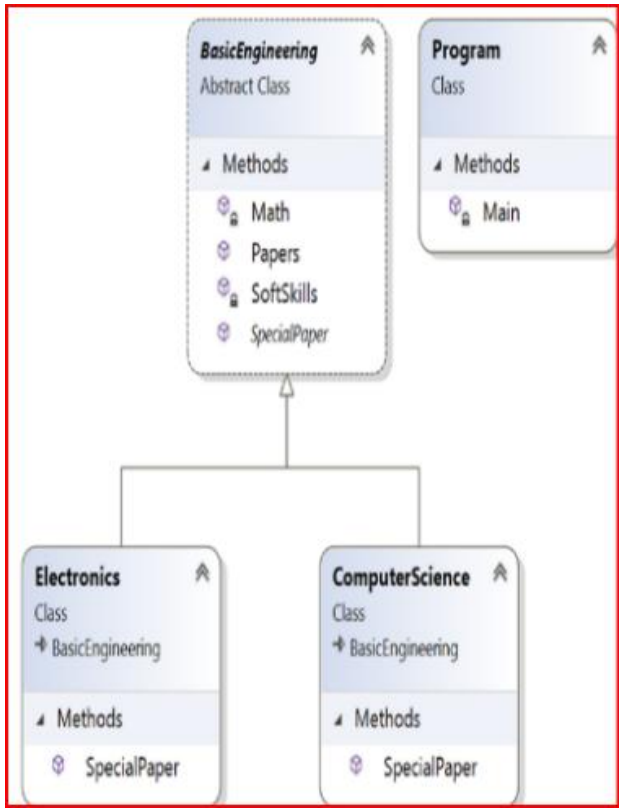
```
public class PenneAlfredo extends
PastaDish {
      protected void addPasta() {
            System.out.println("Add
            penne");
      }
      protected void addProtein() {
            System.out.println("Add
            chicken");
      }
      protected void addSauce() {
            System.out.println("Add
            Alfredo sauce");
      }
      protected void addGarnish() {
            System.out.println("Add
            parsley");
      }
}
```

Usage

- Multiple classes with same functional operations with different implementations and same order of executions.
- Practical application of generalization and inheritance
- Popular in framework development
- Avoid code duplication
- Code Reusability



In this pattern, subclasses can simply redefine the methods based on their needs



In the abstract class BasicEngineering, only one method is abstract (SpecialPaper), and the other two methods are concrete methods. What is the reason behind this?

Answer: This is a simple example with only three methods, and you want the subclasses to override only the SpecialPaper() method here. Other methods are common to both courses, and they do not need to be overridden by the subclasses

Advantages of using a Template Method Design Pattern

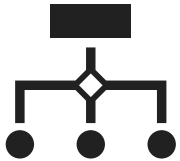


You can control the flow of the algorithms. Clients cannot change them.



Common operations will be in a centralized location. For example, in an abstract class, the subclasses can redefine only the varying parts so that you can avoid redundant code.

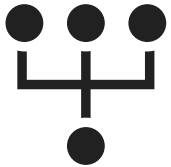
Challenges Associated with a Template Method Design Pattern



The client code cannot direct the sequence of steps. If you want that type of functionality, use the Builder pattern.



A subclass can override a method defined in the parent class (in other words, hiding the original definition in the parent class)



Having more subclasses means more scattered code, which is difficult to maintain.

Reflective Questions



How does the Template Method pattern balance reusability and customizability when defining an algorithm's structure?



In what situations might using the Template Method lead to rigidity or overgeneralization, and how can this be avoided?



How does the Template Method pattern encourage code consistency across subclasses while still allowing subclass-specific behavior?